Differentially Private Query Release for Telemetry Data

Reva Agrawal ragrawal@ucsd.edu
Jordan Lambino jlambino@ucsd.edu
Dhruv Patel dhp005@ucsd.edu
Yu-Xiang Wang yuxiangw@ucsd.edu
Bijan Arbab barbab@ucsd.edu

Abstract

Telemetry from software and hardware systems is vital for product analytics and quality monitoring, but releasing aggregate statistics can expose individuals to re-identification even when identifiers are removed. We address this with a rigorous, practical approach: a differentially private query-release system that answers a fixed set of aggregate queries by adding calibrated noise to query outputs after bounding each device's contribution. We apply the Laplace and Analytic Gaussian mechanisms to 12 benchmark queries over Intel telemetry, covering battery usage, display and browser statistics, power and sleep metrics, and process rankings; a total privacy budget is allocated across queries. Evaluated on mini and full databases, Laplace matches or outperforms Gaussian at strong privacy, and at the selected ϵ most queries pass on the full database. The pipeline is reproducible and provides an auditable (ϵ, δ) guarantee, supporting product and quality teams without access to raw user-level data.

Code: https://github.com/agrawalreva/intel-telemetry-capstone

Website: https://agrawalreva.github.io/intel-telemetry-capstone/

Table of Contents

1 Introduction **4**

1.1 Motivation 4

1.2 Differential Privacy 5

1.3 Telemetry Data 7

1.4 Applications 7

1.5 Related Work 7

2 Methods **8**

2.1 Data Access and Reporting Layer 8

2.2 Project and Pipeline Structure 8

2.3 Mechanism Variants: Baseline and Advance 8

2.4 Global Privacy Model 9

2.5 Evaluation Metrics 10

2.6 Query 1: Battery Power On Geographic Summary 11

2.7 Query 2: Battery On Duration by CPU Family and Generation 12

2.8 Query 3: Display Devices Connection Type Resolution Durations AC/DC 13

2.9 Query 4: Display Devices Vendors Percentage 13

2.10 Query 5: MODS Blockers by OS Name and Codename 14

2.11 Query 6: Most Popular Browser in Each Country by System Count 14

2.12 Query 7: On/Off MODS Sleep Summary by CPU Market Codename and Generation . 15

2.13 Query 8: Persona Web Category Usage Analysis 15

2.14 Query 9: Package Power by Country 16

2.15 Query 10: Popular Browsers by Count Usage Percentage 17

2.16 Query 11: RAM Utilization Histogram 17

2.17 Query 12: Ranked Process Classifications 18

2.18 Privacy Budget Allocation 18

2.19 Multiple Epsilon Evaluation and the Privacy-Utility Curve 19

3 Results **19**

3.1 Experimental Setup . . . 19
3.2 Privacy Parameters . . . 19
3.3 Utility Metrics and Pass Rates . . . 19
3.4 Laplace vs Gaussian Comparison . . . 21
3.5 Best Epsilon Selection . . . 23
3.6 Full-Database Results at $\epsilon = 1.0$ . . . 23
3.7 Visualization Pipeline: Mini vs Full Datasets . . . 24
3.8 Privacy-Utility Figures . . . 24
4 Discussion . . . 29
4.1 Privacy and Utility Tradeoff . . . 29
4.2 Interpretation of the Privacy-Utility Tradeoff . . . 29
4.3 Comparison with Alternative Approach . . . 29
4.4 Practical Implications . . . 30
4.5 Scope and Future Directions . . . 30
4.6 Assumptions and Threat Model . . . 30
4.7 Deployment Considerations . . . 30
5 Conclusion . . . 31
5.1 Summary . . . 31
5.2 Impact . . . 31
5.3 Future Work . . . 32
5.4 Acknowledgements . . . 32
References . . . 32
Appendices . . . A1
A.1 Appendix A: Contributions . . . A1
A.2 Appendix B: Project Proposal . . . A2
A.3 Appendix C: Additional Materials . . . A2

1 Introduction

Hardware and software vendors such as Intel rely on telemetry to understand how products behave in the field. They need aggregate statistics on battery usage, display and power metrics, adoption of features and configurations, and failure or performance patterns by geography, device type, and workload. These insights drive quality improvement, support prioritization, and product planning. To obtain them, vendors collect event logs and system attributes from participating devices: each record is associated with a device identifier (typically a GUID), and data is shipped to central systems for aggregation and analysis. The resulting datasets are large, high-dimensional, and user-level, which makes them valuable for analytics but also sensitive. Releasing aggregate statistics, even after stripping direct identifiers, can still enable re-identification or linkage when combined with other information. The tension between the need for actionable insights and the obligation to protect individuals is the problem we address.

In this work we apply *differentially private query release* to Intel telemetry: we answer a fixed set of analytical queries by adding calibrated noise to their outputs after bounding each device's contribution via clipping, and we evaluate two standard mechanisms (Laplace and Analytic Gaussian) on mini and full databases.

1.1 Motivation

Organizations that collect telemetry at scale need aggregate statistics for collaboration, benchmarking, and research while protecting individual users. Query-based release under Differential Privacy gives analysts noisy answers to predefined queries with a formal guarantee that the probability of any output changes by at most a factor e^{ϵ} (and an additive δ when applicable) when one user's data is added or removed. Implementing this in practice requires defining the unit of privacy (here, one GUID), deriving and bounding each query's global sensitivity, choosing a mechanism and calibrating noise to that sensitivity, and accounting for composition when multiple queries are answered. An alternative would be private synthetic data generation: learn a DP model of the full data distribution, generate a synthetic dataset, and release it so analysts can run arbitrary queries without further privacy cost. That approach offers flexibility and a one-time budget, but is less suited to our setting: the telemetry is large, high-dimensional, and sparse, and many queries join dozens of columns. Modelling the full joint distribution under DP is extremely demanding at that scale, and synthetic data tends to over-smooth, miss rare but important patterns, or consume a large budget for the whole distribution instead of the summaries we need. Because we have a set of 12 well-defined aggregate queries and the use case is enterprise reporting, we adopt query release instead, spending the privacy budget only on those 12 answers, bounding sensitivity tightly per GUID and per query, and allocating epsilon by importance. That yields better accuracy for the statistics we actually use and keeps the guarantee auditable and the implementation tractable at scale.

1.2 Differential Privacy

Differential privacy (DP) is a mathematical definition of what it means for a randomized computation to limit the harm to an individual from participating in a dataset. The idea, due to Dwork and colleagues Dwork et al. (2006); Dwork and Roth (2014), is that the *distribution* of the mechanism's output should change only slightly when one person's data is added or removed. Thus, an adversary who sees the output cannot confidently tell whether any one individual was present, so participation carries bounded privacy risk. Formally, a mechanism $\mathscr{M}$ is differentially private if for every pair of datasets that differ in one person's records, and for every subset S of possible outputs, the probability $\Pr[\mathscr{M}(D) \in S]$ is close to $\Pr[\mathscr{M}(D') \in S]$. The standard way to quantify "close" is with two parameters: $\epsilon \geq 0$ (a multiplicative bound) and $\delta \geq 0$ (an additive slack). The central inequality is

$$\Pr[\mathscr{M}(D) \in S] \;\leq\; e^{\epsilon} \cdot \Pr[\mathscr{M}(D') \in S] \;+\; \delta. \tag{1}$$

When $\delta = 0$, the guarantee is *pure* ϵ-differential privacy: the output distribution changes by at most a factor e^{ϵ} when one person's data is removed. Smaller ϵ and δ mean stronger privacy; in practice ϵ is often chosen in a range from about 0.01 (very strong) to a few units (weaker but more accurate).

Any finite ϵ yields a formal (ϵ, δ)-DP release; only $\epsilon = \infty$ corresponds to no noise. We now make the definition precise and state the sensitivity and mechanisms we use.

Neighboring datasets. We use *user-level* (device-level) differential privacy. Two datasets D and D' are *neighboring*, written $D \sim D'$, if one can be obtained from the other by adding or removing all records belonging to a single GUID. Thus the "one person" whose presence or absence we protect is one device's entire contribution to the dataset.

Formal definition. A randomized mechanism $\mathscr{M}$ with range $\mathscr{R}$ satisfies (ϵ, δ)-*differential privacy* if for all neighboring D, D' and all measurable $S \subseteq \mathscr{R}$,

$$\Pr[\mathscr{M}(D) \in S] \leq e^{\epsilon}\, \Pr[\mathscr{M}(D') \in S] + \delta. \tag{2}$$

When $\delta = 0$, we say $\mathscr{M}$ satisfies *pure* ϵ-DP Dwork and Roth (2014).

Global sensitivity. To add the right amount of noise, we need a bound on how much the *query function* f can change when one person's data changes. The *global sensitivity* of f is the maximum change over all neighboring datasets. For a real-valued f,

$$\Delta f \;=\; \max_{D \sim D'} \big| f(D) - f(D') \big|. \tag{3}$$

For a vector-valued f with values in $\mathbb{R}^d$, we use two norms: the ℓ_1 global sensitivity $\Delta_1 f = \max_{D \sim D'} \sum_{i=1}^{d} |f_i(D) - f_i(D')|$ (for the Laplace mechanism) and the ℓ_2 global sensitivity $\Delta_2 f = \max_{D \sim D'} \|f(D) - f(D')\|_2$ (for the Gaussian mechanism). Sensitivity must be a worst-case,

data-independent bound. For the query types in this work we use: **Count:** $\Delta f = 1$. **Sum with per-GUID clipping at** C**:** $\Delta f = C$. **Average** over at least $k_{\min}$ GUIDs with per-GUID contribution clipped to C: $\Delta f = C/k_{\min}$. **Histogram** (one bin per GUID): each GUID affects at most one bin by 1, so the ℓ_1 sensitivity of the full vector is 1 and the ℓ_2 sensitivity is 1.

Laplace mechanism. For a function f with ℓ_1 global sensitivity $\Delta_1 f$, the Laplace mechanism Dwork et al. (2006); Dwork and Roth (2014) releases

$$\mathscr{M}(D) = f(D) + \mathrm{Lap}(0, b), \qquad b = \frac{\Delta_1 f}{\epsilon}. \tag{4}$$

Here $\mathrm{Lap}(0, b)$ denotes a draw from the Laplace distribution with mean 0 and scale b (so the variance is $2b^2$). This mechanism satisfies *pure* ϵ-DP with no δ term, making it particularly efficient for scalar aggregate queries. For vector-valued f, we use the ℓ_1 sensitivity of the vector and add independent $\mathrm{Lap}(0, \Delta_1 f/\epsilon)$ to each coordinate; the resulting mechanism is ϵ-DP Dwork and Roth (2014).

Gaussian and Analytic Gaussian Mechanism. For (ϵ, δ)-DP, the Gaussian mechanism releases $f(D) + Z$ where $Z \sim \mathcal{N}(0, \sigma^2 I)$. The classical analysis gives a sufficient condition $\sigma \geq (\Delta_2 f/\epsilon)\sqrt{2\ln(1.25/\delta)}$, which is not tight. The *Analytic Gaussian Mechanism* Balle and Wang (2018) calibrates σ via the exact (ϵ, δ)-DP condition. For a scalar output with ℓ_2 sensitivity Δ, the mechanism is (ϵ, δ)-DP if and only if

$$\Phi\left(\frac{\Delta}{2\sigma} - \frac{\epsilon\sigma}{\Delta}\right) - e^{\epsilon} \cdot \Phi\left(-\frac{\Delta}{2\sigma} - \frac{\epsilon\sigma}{\Delta}\right) \leq \delta, \tag{5}$$

where Φ is the standard normal CDF Balle and Wang (2018). We find the smallest σ satisfying this inequality (e.g., by binary search). For vector-valued f, we use the ℓ_2 sensitivity $\Delta_2 f$ in place of Δ. This analytic calibration yields a smaller σ than the classical bound for the same (ϵ, δ), hence better utility.

Composition and post-processing. If $\mathscr{M}_1$ is (ϵ_1, δ_1)-DP and $\mathscr{M}_2$ is (ϵ_2, δ_2)-DP, their sequential composition is $(\epsilon_1 + \epsilon_2, \delta_1 + \delta_2)$-DP (basic composition Dwork and Roth (2014)). We allocate a total budget ϵ_{total} across the 12 queries and state the composed guarantee explicitly (Section 2.18). Post-processing does not weaken the guarantee: if $\mathscr{M}$ is (ϵ, δ)-DP and g is any deterministic function, then $g \circ \mathscr{M}$ is (ϵ, δ)-DP. Hence clamping outputs to non-negative values or re-normalising percentages after adding noise costs no extra privacy.

Report Noisy Max. For queries that release the identity of the maximum over a set of counts (e.g., the most popular browser per country), we use Report Noisy Max Dwork and Roth (2014): add independent $\mathrm{Lap}(0, 1/\epsilon)$ to each count (each count has sensitivity 1), then release the index of the maximum noisy count. The whole selection is ϵ-DP.

1.3 Telemetry Data

The data used in this work is Intel telemetry logs. The source data consists of 23 tables across two schemas. Each row is associated with a device identifier (GUID); multiple rows per GUID are common. The data exhibits heavy-tailed distributions; hence a small fraction of GUIDs can contribute disproportionately to aggregates. We therefore *clip* per-GUID contributions before aggregation so that global sensitivity is bounded by a fixed constant. We cap quantities (including the number of power-on events per GUID and the total duration per GUID in a given category) at constants C_1, C_2, and similar bounds, as specified per query in Section 2. These caps are chosen so that global sensitivity is bounded in the worst case (e.g., when a group has exactly $k_{\min}$ devices) while still covering typical or high-percentile usage (e.g., 600 s for display duration, 720 min for daily sleep time, 10 W for per-user power). The aggregate query is then evaluated on the clipped view and the DP mechanism is applied to the query output. Clipping is a deterministic pre-processing step; global sensitivity is computed with respect to the clipped query function only.

We use a specialized reporting layer of 22 pre-aggregated tables built from the raw tables via a SQL build script, and we run 12 benchmark queries covering battery usage by geography and CPU family, display device vendor market share, browser popularity by country, OS-level MODS blockers, RAM utilization, persona-level web category usage, and process power rankings. The 12 query results are exported as baseline (non-private) CSVs. We then apply both the Laplace and the Analytic Gaussian Mechanism to every query (Section 2.4), with epsilon allocated via sequential composition (Section 2.18). Post-processing consists of clamping numeric outputs to non-negative values where required and re-normalising percentage columns to sum to 100% where applicable. The pipeline is developed and validated on a subsample database (approximately 5 GB) before execution on the full dataset.

1.4 Applications

Privacy-preserving query release over telemetry supports usage metrics and feature adoption by segment, performance and health monitoring by geography or hardware family, market share and ranking statistics, and anomaly detection. Analysts receive noisy answers suitable for trend analysis and prioritisation without access to raw user-level data. The same framework applies to other telemetry or event-log datasets where user-level differential privacy is required.

1.5 Related Work

Differential privacy was introduced in Dwork et al. (2006) and is treated in depth in Dwork and Roth (2014), which covers the Laplace and Gaussian mechanisms, global sensitivity, and composition. The Analytic Gaussian Mechanism of Balle and Wang (2018) improves noise calibration for (ϵ, δ)-DP. Query answering under DP in database and analytics settings requires bounding sensitivity and allocating privacy budget across queries; our guarantees follow from these standard tools and explicit sequential composition.

2 Methods

This section describes how we obtain the data, how the pipeline is structured, the global privacy model (unit of privacy, sensitivity, mechanisms, composition), the choice and meaning of the three evaluation metrics (median relative error, total variation distance, and Spearman rank correlation) and their assignment to queries, the allocation of the privacy budget across queries, and the use of multiple epsilon values to trace the privacy-utility curve. We then give the full mathematical definition, sensitivity, mechanism, and evaluation metric for each of the 12 queries (Sections 2.6 to 2.17).

2.1 Data Access and Reporting Layer

Raw Intel telemetry is stored in 23 tables across two schemas. We build two DuckDB databases: a subsample with capped table sizes for development, and a full database for production. A build step (SQL script `00_build_reporting_tables.sql`) constructs 22 reporting tables that resolve joins and normalise column names. The 12 benchmark queries are executed against this reporting layer; their results are exported and serve as the non-private baseline. Per-query clipping is applied in SQL so that each device's contribution is bounded before aggregation; global sensitivity is derived from these clipping constants and the query structure only.

2.2 Project and Pipeline Structure

The pipeline has six stages. Raw telemetry is loaded into DuckDB and a build step produces 22 reporting tables. The 12 benchmark queries are then run with per-GUID clipping in SQL to obtain the non-private baseline. For the DP release we run the Laplace and Analytic Gaussian mechanism at each ϵ in $\{0.01, 0.05, 0.1, 0.5, 1.0, \infty\}$ (with ∞ as the no-noise reference); two implementation variants are defined in Section 2.3. Utility is quantified by median relative error, total variation distance, and Spearman rank correlation (Section 2.5). Evaluation computes per-query statistics and Laplace vs Gaussian comparison across ϵ. Best-epsilon selection uses a utility-preservation score relative to the $\epsilon = \infty$ baseline: Tier 1 is the smallest ϵ where mean utility preserved reaches 80%, with a fallback to the ϵ with highest score otherwise. Finally, privacy-utility curves, pass rate, and Pareto frontier are produced from the evaluation outputs.

2.3 Mechanism Variants: Baseline and Advance

The pipeline implements each mechanism in two variants: **baseline** and **advance**. They are *mathematically identical* (same noise distribution and privacy guarantee); the only difference is how the noise scale is computed at run time.

In the **baseline** variant the scale is recomputed for every (query, column, ϵ) triple: $b =$

$\Delta_1 f/\epsilon$ for Laplace, and σ via binary search for the Analytic Gaussian Balle and Wang (2018). In the **advance** variant a *scale cache* is built once per (sensitivity, ϵ) pair and looked up during release; this reduces runtime (often 2–4× faster) without changing outputs or guarantees. The advance variant typically attains similar utility at a lower ϵ (see Section 4.2).

2.4 Global Privacy Model

Unit of privacy. We use *user-level* differential privacy: the unit of privacy is one GUID. Neighboring datasets D and D' differ in all rows associated with exactly one GUID (i.e., adding or removing one user's entire activity).

Contribution bounding. Before aggregation we apply per-GUID contribution clipping so that the influence of any single GUID on the query output is bounded. Clipping is a deterministic, data-independent pre-processing step; global sensitivity is computed with respect to the clipped query function.

Sensitivity. For each query we derive the global sensitivity Δf (or $\Delta_1 f$, $\Delta_2 f$ for vector outputs) from the clipping bounds and the query structure.

Mechanisms. We run *both* the Laplace and the Analytic Gaussian Mechanism on every query so that we can compare their privacy-utility tradeoff in the evaluation. The per-query "assignment" in the table and subsections indicates which mechanism is *preferred* for that query type (because it typically yields better utility for the same privacy budget), not which one we run. Laplace is preferred for count queries, histograms, Report Noisy Max, and low-dimensional aggregates. The Analytic Gaussian Mechanism is preferred for high-dimensional or multi-column outputs (Queries 7, 8, 9) because its analytic calibration yields a smaller σ than the classical bound for the same (ϵ, δ), improving utility when the budget is spread over many dimensions. Specifically:

- **Laplace mechanism** (pure ϵ-DP): release $f(D) + \mathrm{Lap}(0, b)$ with $b = \Delta_1 f/\epsilon$.
- **Analytic Gaussian Mechanism** ((ϵ, δ)-DP): release $f(D) + \mathcal{N}(0, \sigma^2)$ where σ is calibrated via Balle and Wang (2018). We fix $\delta = 10^{-6}$ per query.

Composition. The 12 queries are answered under *sequential composition*: if query i uses (ϵ_i, δ_i)-DP, the total guarantee is $(\sum_{i=1}^{12} \epsilon_i, \sum_{i=1}^{12} \delta_i)$-DP. We allocate a total budget ϵ_{total} so that $\sum_{i=1}^{12} \epsilon_i \leq \epsilon_{\text{total}}$. When a single query releases multiple numeric outputs, we either split that query's ϵ_i across outputs or treat the output as a vector and apply the vector-valued mechanism once.

Post-processing. Deterministic post-processing (clamping to non-negative, re-normalising percentages to sum to 100%) does not consume privacy budget and preserves (ϵ, δ)-DP.

2.5 Evaluation Metrics

Utility is evaluated by comparing the true (non-private) query output with the DP output. We assign each query exactly one of three metrics, aligned with the shared evaluation framework: **median relative error**, **total variation distance**, and **Spearman rank correlation**. The metric for each query is chosen to match the analytical story (aggregate deviation, distribution shape, or ranking). Table 1 summarises the three metrics and their assignment to queries.

Table 1: Evaluation metrics: name, what it measures, pass threshold, and query assignment.

Metric	Name	What it measures	Pass	Queries
Median rel. err.	Median relative error	Noisy aggregate deviation from true	≤ 0.25	Q1, Q2, Q4, Q5, Q9
TVD	Total variation distance	Distortion of percentage/distribution	≤ 0.15	Q7, Q8, Q10, Q11
Spearman ρ	Spearman rank correlation	Ranking preserved	$\rho \geq 0.5$	Q3, Q6, Q12

Median relative error. Queries Q1, Q2, Q4, Q5, and Q9: we compute the median of $|x_i^{\text{true}} - x_i^{\text{DP}}|/|x_i^{\text{true}}|$ over groups (or the key column). Lower is better; pass threshold median relative error ≤ 0.25.

Total variation distance. Queries Q7, Q8, Q10, and Q11: we normalise the true and DP vectors to probability distributions and compute

$$\text{TVD} = \frac{1}{2}\sum_i |p_{\text{true},i} - p_{\text{DP},i}|.$$

Lower is better; 0 means identical distributions. For Q8 (persona-level category distributions) we report the mean total variation distance across personas. Pass threshold total variation distance ≤ 0.15.

Spearman rank correlation. Queries Q3, Q6, and Q12: we compute the Spearman rank correlation between the true and DP orderings (e.g., display configs by duration, browser winner per country, users by power). Higher is better; pass threshold $\rho \geq 0.5$.

Table 2 gives an overview of all 12 queries, their research question, assigned metric, mechanism, and global sensitivity Δ, so that the reader can locate the full derivation in the corresponding subsection.

Table 2: Overview of the 12 benchmark queries: research question, evaluation metric, mechanism, and global sensitivity.

Q	Metric	Research question	Mechanism	Δ
1	Median rel. err.	Battery duration by country (abnormal?)	Laplace	$C/k_{\min}$
2	Median rel. err.	Battery duration by CPU (abnormal?)	Laplace	$C/k_{\min}$
3	Spearman ρ	Display configs ranked by AC duration	Laplace	$C/k_{\min}$
4	Median rel. err.	Vendor percentage distribution	Laplace	1.0
5	Median rel. err.	MODS blockers by OS (abnormal?)	Laplace	$C/k_{\min}$
6	Spearman ρ	Most popular browser per country	Report Noisy Max	1
7	TVD	MODS sleep time by CPU (abnormal?)	Lap. or Gauss.	$C/k_{\min}$
8	TVD	Persona web category distribution (28 dim.)	Lap. or Gauss.	$\epsilon_8/28$ per col.
9	Median rel. err.	Package power by country (abnormal?)	Lap. or Gauss.	$C/k_{\min}$
10	TVD	Browser market share (percent)	Laplace	1.0
11	TVD	RAM utilisation histogram	Laplace	$C/k_{\min}$
12	Spearman ρ	Users ranked by power consumption	Laplace	C (sum)

2.6 Query 1: Battery Power On Geographic Summary

Source tables: reporting.system_batt_dc_events (guid, num_power_ons, duration_mins); reporting.system_sysinfo_unique_normalized (guid, countryname_normalized).

Research question. Which countries have abnormally long battery discharge durations?

Non-private mathematical expression. Let D be the dataset of per-GUID battery discharge events. For each country c, define the clipped per-GUID average of duration with $C = 60$ minutes:

$$x_g^c = \text{clip}(\text{avg}_{\text{session}}(\text{duration_mins}_g), C), \qquad f(D)_c = \frac{1}{n_c}\sum_{g \in c} x_g^c, \quad n_c = |\{g : \text{country}(g) = c\}|,$$

restricted to groups with $n_c \geq k_{\min} = 100$. Output: a vector of (country, avg_duration) pairs, ordered by avg_duration.

Contribution bounding. For each GUID: clip total `num_power_ons` to $C_1 = 10$ and total `duration_mins` to $C_2 = 60$. Averages use $k_{\min} = 100$.

Sensitivity. Adding or removing one GUID changes $f(D)_c$ by at most C/n_c. Worst case over neighboring datasets is $n_c = k_{\min}$, so *global* sensitivity is $\Delta = C/k_{\min} = 60/100 = 0.60$ (we use only this; no data-dependent reduction). Per country we release one scalar (avg_duration); Laplace scale $b = \Delta/\epsilon_1$.

Mechanism: Laplace: add $\mathrm{Lap}(0, \Delta/\epsilon_1)$ to avg_duration per country. Single draw per output. Epsilon: ϵ_1 (medium).

Evaluation metric: Median relative error (on avg_duration).

Post-processing: Clamp ≥ 0; ranking and z-scores are deterministic functions of the noisy output.

2.7 Query 2: Battery On Duration by CPU Family and Generation

Source tables: reporting.system_cpu_metadata (guid, marketcodename, cpugen); reporting.system_batt_dc_events (guid, duration_mins).

Research question. Which CPU family-generation combinations have abnormal battery duration?

Non-private mathematical expression. For each group $c = (\text{marketcodename}, \text{cpugen})$ with $C = 60$:

$$x_g^c = \mathrm{clip}(\mathrm{avg}_{\mathrm{event}}(\mathrm{duration_mins}_g), C), \qquad f(D)_c = \frac{1}{n_c} \sum_{g \in c} x_g^c, \quad n_c \geq k_{\min} = 100.$$

Output ordered by avg_duration.

Contribution bounding. Clip each GUID's total `duration_mins` to $C = 60$.

Sensitivity. One GUID changes the group average by at most C/n_c; worst case $n_c = k_{\min}$ gives $\Delta = C/k_{\min} = 60/100 = 0.60$. Laplace scale $b = \Delta/\epsilon_2$.

Mechanism: Laplace: add $\mathrm{Lap}(0, \Delta/\epsilon_2)$ to avg_duration per group. Single draw. Epsilon: ϵ_2 (medium).

Evaluation metric: Median relative error (on avg_duration).

Post-processing: Clamp ≥ 0.

2.8 Query 3: Display Devices Connection Type Resolution Durations AC/DC

Source tables: reporting.system_display_devices (guid, connection_type, resolution_height, resolution_width, duration_ac, duration_dc).

Research question. Which display connection-type and resolution combinations have the longest AC usage time?

Non-private mathematical expression. For each group $c = (\text{connection_type}, \text{resolution})$ with $C = 600$ seconds (10 minutes):

$$x_g^c = \text{clip}(\text{avg}_{\text{session}}(\text{duration_ac}_g), C), \qquad f(D)_c = \frac{1}{n_c} \sum_{g \in c} x_g^c, \quad n_c \geq k_{\min} = 50.$$

Output ranked by $f(D)_c$ descending.

Contribution bounding. Clip per-GUID `duration_ac` and `duration_dc` to $C = 600$ seconds.

Sensitivity. One GUID changes the group average by at most C/n_c; worst case $n_c = k_{\min}$ gives $\Delta = C/k_{\min} = 600/50 = 12.0$ seconds. Laplace scale $b = \Delta/\epsilon_3$ for the released average_duration_on_ac.

Mechanism: Laplace: add $\text{Lap}(0, \Delta/\epsilon_3)$ to average_duration_on_ac per group. Single draw. Epsilon: ϵ_3 (smaller).

Evaluation metric: Spearman ρ (rank correlation on average_duration_on_ac).

Post-processing: Clamp ≥ 0; ranking is deterministic post-processing.

2.9 Query 4: Display Devices Vendors Percentage

Source tables: reporting.system_display_devices (guid, vendor_name).

Research question. What fraction of systems use each display vendor?

Non-private mathematical expression. Let $N = \sum_v n_v$ be the total number of distinct GUIDs. For each vendor v:

$$n_v = |\{g : \text{vendor}(g) = v\}|, \qquad f(D)_v = (n_v/N) \times 100 \text{ (percentage of systems)}.$$

Output: vector (vendor, percentage) summing to 100%.

Contribution bounding. One GUID contributes to exactly one vendor. We add noise *directly to the percentage column* (the baseline export already contains percentages computed from counts). The true sensitivity of a percentage to one GUID is $O(1/N)$; we use a deliberate conservative choice $\Delta = 1.0$ (one percentage point on the 0–100 scale) so that the guarantee holds regardless of N. This choice drives higher median relative error (e.g., 0.66 on full) and could be relaxed in future work (e.g., noising counts then forming percentages).

Sensitivity. $\Delta = 1.0$. Laplace scale $b = \Delta/\epsilon_4$ per vendor percentage.

Mechanism: Laplace: add $\text{Lap}(0, \Delta/\epsilon_4)$ to each vendor's percentage. Single draw. Epsilon: ϵ_4 (smaller).

Evaluation metric: Median relative error (on percentage_of_systems).

Post-processing: Clamp ≥ 0; re-normalise to sum to 100% (deterministic).

2.10 Query 5: MODS Blockers by OS Name and Codename

Source tables: reporting.system_mods_top_blocker_hist (guid, dt); reporting.system_os_codename_hist (guid, min_ts, max_ts, os_name, os_codename). Join on GUID and time windows.

Research question. Which OS versions generate the most MODS blocker events per system?

Non-private mathematical expression. For each group $c = (\text{os_name}, \text{os_codename})$ with $C = 50$:

$$e_g^c = \text{clip}(\text{COUNT}(\text{events for } g \in c), C),$$

$$f(D)_c = \frac{1}{n_c} \sum_{g \in c} e_g^c \text{ (avg. clipped entries per system)}, \quad n_c \geq k_{\min} = 10.$$

Contribution bounding. Clip each GUID's entry count to $C = 50$ per group.

Sensitivity. One GUID changes the group average by at most C/n_c; worst case $n_c = k_{\min}$ gives $\Delta = C/k_{\min} = 50/10 = 5.0$. Laplace scale $b = \Delta/\epsilon_5$.

Mechanism: Laplace: add $\text{Lap}(0, \Delta/\epsilon_5)$ to entries_per_system per group. Single draw. Epsilon: ϵ_5 (smaller).

Evaluation metric: Median relative error (on entries_per_system).

Post-processing: Clamp ≥ 0.

2.11 Query 6: Most Popular Browser in Each Country by System Count

Source tables: reporting.system_sysinfo_unique_normalized (guid, countryname_normalized); reporting.system_web_cat_usage (guid, browser).

Research question. Which browser is most used in each country?

Non-private mathematical expression. For each country c and browser b:

$$\text{count}(c, b) = |\{g : \text{country}(g) = c, \text{browser}(g) = b\}|, \qquad f(D)_c = \arg\max_b \text{count}(c, b).$$

Output: one (country, browser) pair per country.

Sensitivity. Each $\text{count}(c, b)$ has global sensitivity $\Delta = 1$ (one GUID changes one count by 1).

Mechanism: Report Noisy Max. Per country: add $\mathrm{Lap}(0, 1/\epsilon_6)$ to each browser count; release $\arg\max_b(\mathrm{count}(c, b) + \mathrm{noise}_b)$. This satisfies ϵ_6-DP per country. Both the "Laplace" and "Gaussian" pipeline runs use this same Report Noisy Max (Laplace noise on the count); the two rows for Q6 in the evaluation table are the two runs, not two different mechanisms.

Evaluation metric: Spearman ρ (rank correlation on winner ordering per country).

Post-processing: None beyond argmax.

2.12 Query 7: On/Off MODS Sleep Summary by CPU Market Codename and Generation

Source tables: reporting.system_on_off_suspend_time_day (guid, on_time, off_time, mods_time, sleep_time); reporting.system_cpu_metadata (guid, marketcodename, cpugen).

Research question. Which CPU families spend an abnormally high fraction of time in Modern Standby (MODS)?

Non-private mathematical expression. For each group $c = (\text{marketcodename}, \text{cpugen})$ with $C = 720$ minutes per day (12 hours):

$$x_g^c = \mathrm{clip}(\mathrm{avg}_{\mathrm{day}}(\mathrm{mods_time}_g), C), \qquad f(D)_c = \frac{1}{n_c} \sum_{g \in c} x_g^c \ (\mathrm{avg_modern_sleep_time}), \quad n_c \geq k_{\min} = 100.$$

Output includes multiple time columns; the evaluation metric uses only avg_modern_sleep_time.

Contribution bounding. Clip per-GUID total time per category to $C = 720$ minutes per day. For the key column (avg_modern_sleep_time): $\Delta = C/k_{\min} = 720/100 = 7.2$ minutes. We add noise to *every* released numeric column (or treat the release as one vector with joint L1/L2 sensitivity and one mechanism call).

Sensitivity. $\Delta = 7.2$ for the average; if releasing multiple columns, joint sensitivity is used (e.g., L2 over the vector of sensitivities).

Mechanism: Laplace or Analytic Gaussian Mechanism: add noise with scale Δ/ϵ_7 (Laplace) or $\sigma(\Delta, \epsilon_7, \delta)$ (Gaussian) to the released statistics. Single draw. Epsilon: ϵ_7 (medium).

Evaluation metric: Total variation distance (mean over sleep percentage columns).

Post-processing: Clamp ≥ 0; re-normalise percentages if computed.

2.13 Query 8: Persona Web Category Usage Analysis

Source tables: reporting.system_sysinfo_unique_normalized (guid, persona); reporting.system_web_cat_pivot_duration (guid, category duration columns).

Research question. How does web browsing time distribute across 28 content categories for each user persona?

Non-private mathematical expression. For each persona p and category k: $C_{\text{days}} = 365$, $C_{\text{dur}} = 3{,}600{,}000$ ms;

$$\text{days}_g = \text{clip}(\text{COUNT}(\text{days for } g), C_{\text{days}}), \quad \text{cat}_g^k = \text{clip}(\text{SUM}(\text{category}_k \text{ duration for } g), C_{\text{dur}}),$$

$$f(D)_{p,k} = \sum_{g \in p} \text{days}_g \cdot (\text{cat}_g^k / \text{total}_g) \Big/ \sum_{g \in p} \text{days}_g.$$

Output: 28-dimensional (fraction or percentage) vector per persona, summing to ∼100%.

Contribution bounding. Per GUID, each category duration is clipped and total_g is the sum of all category durations; hence $\text{cat}_g^k/\text{total}_g \in [0, 1]$ by construction ($\text{total}_g \geq \text{cat}_g^k$). Thus each category output has sensitivity $\Delta \leq 1$ per column in normalised form. We use the **split-budget** approach: allocate $\epsilon_{\text{per column}} = \epsilon_8/28$ per column so the total composition is ϵ_8. Both Laplace and Analytic Gaussian are implemented with this split (not the single vector release with ℓ_2 sensitivity $\sqrt{28}$).

Sensitivity. Per column: $\Delta_j \leq 1$, $\epsilon_j = \epsilon_8/28$.

Mechanism: Laplace or Analytic Gaussian with $\epsilon_8/28$ per column. Single draw per output per column.

Evaluation metric: Total variation distance (mean across persona category distributions).

Post-processing: Clamp ≥ 0; re-normalise each persona's row to sum to 100%; optional Dirichlet smoothing to avoid zeros in KL.

2.14 Query 9: Package Power by Country

Source tables: reporting.system_sysinfo_unique_normalized (guid, countryname_normalized); reporting.system_hw_pkg_power (guid, nrs, mean).

Research question. Which countries have abnormally high CPU package power consumption?

Non-private mathematical expression. For each country c with $C = 50$ W:

$$x_g^c = \text{clip}(\text{SUM}(\text{nrs} \cdot \text{mean})/\text{SUM}(\text{nrs}) \text{ for } g, C), \qquad f(D)_c = \frac{1}{n_c} \sum_{g \in c} x_g^c, \quad n_c \geq k_{\min} = 100.$$

Contribution bounding. Clip per-GUID average power to $C = 50$ W.

Sensitivity. One GUID changes the group average by at most C/n_c; worst case $n_c = k_{\min}$ gives $\Delta = C/k_{\min} = 50/100 = 0.50$ W. Laplace scale $b = \Delta/\epsilon_9$.

Mechanism: Laplace or Analytic Gaussian Mechanism: add noise with scale Δ/ϵ_9. Single draw. Epsilon: ϵ_9 (medium).

Evaluation metric: Median relative error (on avg_pkg_power_consumed).

Post-processing: Clamp ≥ 0.

2.15 Query 10: Popular Browsers by Count Usage Percentage

Source tables: reporting.system_web_cat_usage (guid, browser, duration_ms).

Research question. What are the global market shares of browsers by system count (and optionally instance count, duration)?

Non-private mathematical expression. For each browser b:

$$n_b = |\{g : \text{browser}(g) = b\}|, \quad N = \sum_b n_b, \qquad f(D)_b = (n_b/N) \times 100 \text{ (percent_systems)}.$$

Noise is added directly to the percentage columns (same conservative $\Delta = 1.0$ as for Q4).

Sensitivity. $\Delta = 1.0$ (percentage point). Laplace scale $b = \Delta/\epsilon_{10}$.

Mechanism: Laplace: add $\text{Lap}(0, \Delta/\epsilon_{10})$ to percent_systems. Single draw. Epsilon: ϵ_{10} (smaller).

Evaluation metric: Total variation distance (on percent_systems distribution).

Post-processing: Clamp ≥ 0; re-normalise to sum to 100%.

2.16 Query 11: RAM Utilization Histogram

Source tables: reporting.system_memory_utilization (guid, sysinfo_ram, nrs, avg_percentage_used).

Research question. How does average RAM utilisation distribute across RAM size bins?

Non-private mathematical expression. For each RAM bin b (e.g., ram_gb value) with $C = 100\%$:

$$x_g^b = \text{clip}(\text{avg}_{\text{measurement}}(\text{pct_used}_g), C), \qquad f(D)_b = \frac{1}{n_b} \sum_{g \in b} x_g^b, \quad n_b \geq k_{\min} = 50.$$

Output: (ram_gb, avg_percentage_used) per bin.

Contribution bounding. Each GUID contributes to one bin; per-GUID avg percentage is in $[0, 100]$, clipped for robustness.

Sensitivity. One GUID changes one bin's average by at most C/n_b; worst case $n_b = k_{\min}$ gives $\Delta = C/k_{\min} = 100/50 = 2.0$ percentage points. Laplace scale $b = \Delta/\epsilon_{11}$ per bin.

Mechanism: Laplace: add $\text{Lap}(0, \Delta/\epsilon_{11})$ to avg_percentage_used per bin. Single draw. Epsilon: ϵ_{11} (smaller).

Evaluation metric: Total variation distance (on avg_percentage_used histogram, normalised to a distribution).

Post-processing: Clamp ≥ 0 (bins are independent; no re-normalisation required if releasing means).

2.17 Query 12: Ranked Process Classifications

Source tables: reporting.system_mods_power_consumption (user_id, total_power_consumption).

Research question. Which users contribute the largest total CPU power consumption across all recorded processes?

Non-private mathematical expression. For each user u with $C = 10$ (watts):

$$f(D)_u = \text{clip}\Big(\sum_{\text{process}} \text{power_consumption}_{u,\text{process}}, C\Big).$$

Output: ranked list of (user_id, total_power_consumption) by $f(D)_u$ descending. In practice the identifiers would correspond to anonymised device IDs or process categories rather than directly identifiable users. This is a SUM query rather than an average. Removing a single user from the dataset changes only that user's total power value, and the maximum change is bounded by the clipping constant C.

Sensitivity. $\Delta = C = 10$ per user (global sensitivity of the sum). Laplace scale $b = \Delta/\epsilon_{12}$ per user. Composition: one vector release over all users; total ϵ_{12} for the query.

Mechanism: Laplace: add $\text{Lap}(0, \Delta/\epsilon_{12})$ to total_power_consumption per user. Single draw. Epsilon: ϵ_{12} (moderate).

Evaluation metric: Spearman ρ (rank correlation on total_power_consumption). On the mini database, $\rho = 0$ across ϵ is expected: the Laplace noise scale is $b = 10$ at $\epsilon = 1.0$, which is large relative to the range of observed total power values (roughly 0–100), so the added noise can substantially change the ordering of users, leading to low Spearman rank correlation.

Post-processing: Clamp ≥ 0; ranking is deterministic post-processing.

2.18 Privacy Budget Allocation

We fix a total privacy budget ϵ_{total} and allocate per-query budgets $\epsilon_1, \ldots, \epsilon_{12}$ such that

$$\sum_{i=1}^{12} \epsilon_i \leq \epsilon_{\text{total}}.$$

The allocation reflects the sensitivity and dimensionality of each query: Query 8 (28-dimensional) receives a larger share (about 20% of ϵ_{total}), with ϵ_8 consumed by splitting $\epsilon_8/28$ per category column (Section 2.13). Core usage queries Q1, Q2, Q7, Q9 receive a medium allocation; simple histograms and percentages (Q4, Q5, Q10, Q11) receive a smaller allocation; ranking and Report Noisy Max (Q6, Q12) receive a moderate allocation; Q3 receives a smaller allocation. For the Analytic Gaussian Mechanism we fix $\delta = 10^{-6}$ per query. Under sequential composition the full system satisfies $(\epsilon_{\text{total}}, \delta_{\text{total}})$-DP with $\epsilon_{\text{total}} = \sum_{i=1}^{12} \epsilon_i$ and $\delta_{\text{total}} = \sum_{i=1}^{12} \delta_i = 12 \times 10^{-6}$.

2.19 Multiple Epsilon Evaluation and the Privacy-Utility Curve

We run at $\epsilon \in \{0.01, 0.05, 0.1, 0.5, 1.0, \infty\}$ to trace the privacy-utility curve and select an operating point for full-database runs (Section 2.2). At each ϵ we compute the evaluation metrics and compare Laplace vs Gaussian; $\epsilon = \infty$ is the no-noise reference.

3 Results

We report results from the mini (subsample) and full databases for both the baseline and advance variants (Section 2.3).

3.1 Experimental Setup

Experiments were run on **mini** (subsample, ∼5 GB) and **full** (production-scale), using the ϵ grid and both mechanisms (Section 2.19) with a fixed random seed.

3.2 Privacy Parameters

For the Laplace mechanism we use pure ϵ-DP ($\delta = 0$). For the Analytic Gaussian Mechanism we set $\delta = 10^{-6}$ per query; under sequential composition across 12 queries the full system satisfies $(\epsilon_{\text{total}}, 12 \times 10^{-6})$-DP. Sensitivity per query is derived from the clipping constants and $k_{\min}$ (e.g., $\Delta = C/k_{\min}$ for averages, $\Delta = C$ for sums, $\Delta = 1$ for counts and conservative percentage columns).

3.3 Utility Metrics and Pass Rates

Utility is measured per query by the metric type assigned in Section 2.5: median relative error for queries 1, 2, 4, 5, and 9; total variation distance for queries 7, 8, 10, and 11; Spearman rank correlation for queries 3, 6, and 12. Pass thresholds are median relative error ≤ 0.25, total variation distance ≤ 0.15, and Spearman $\rho \geq 0.5$.

Tables 3 and 4 report the per-query evaluation summary for the baseline and advance variants. The worst-value column shows the highest error (or lowest rank correlation) observed at any ϵ; the best-value column shows performance at $\epsilon = \infty$ (no noise). The pass rate column counts how many of the 6 tested ϵ values (excluding ∞) meet the pass threshold.

Two queries merit a brief note. Q6 (most popular browser per country) uses Report Noisy Max; both Laplace and Gaussian runs apply this same mechanism, so the two table rows correspond to the two pipeline runs. Q12 (ranked processes by power) shows $\rho = 0$ on mini because the Laplace scale $b = 10$ at $\epsilon = 1.0$ is large relative to the range of total power values (roughly 0–100), so the added noise substantially changes the user ordering and yields low Spearman correlation (Section 2.17).

Table 4: Advance variant. Columns as in Table 3.

Baseline variant (Table 3).

Q	Mech.	Metric	Worst val.	Mean	Std	Pass rate
Q1	Gaussian	RE	1.000	0.290	0.385	4/6
Q1	Laplace	RE	0.848	0.188	0.330	5/6
Q2	Gaussian	RE	1.000	0.301	0.386	4/6
Q2	Laplace	RE	1.000	0.226	0.388	5/6
Q3	Gaussian	Spearman	0.201	0.806	0.312	5/6
Q3	Laplace	Spearman	0.472	0.894	0.209	5/6
Q4	Gaussian	RE	1.000	0.833	0.408	1/6
Q4	Laplace	RE	1.000	0.828	0.406	1/6
Q5	Gaussian	RE	18.165	4.212	7.006	1/6
Q5	Laplace	RE	6.763	1.611	2.575	2/6
Q6	Gaussian	Spearman	0.007	0.007	0.000	0/6
Q6	Laplace	Spearman	0.007	0.007	0.000	0/6
Q7	Gaussian	TVD	0.693	0.317	0.277	3/6
Q7	Laplace	TVD	0.634	0.233	0.248	3/6
Q8	Gaussian	TVD	0.576	0.372	0.216	1/6
Q8	Laplace	TVD	0.558	0.295	0.202	1/6
Q9	Gaussian	RE	1.000	0.542	0.483	3/6
Q9	Laplace	RE	1.000	0.325	0.396	3/6
Q12	Gaussian	Spearman	0.000	0.000	0.000	0/6
Q12	Laplace	Spearman	0.000	0.000	0.000	0/6

Advance variant (Table 4).

Q	Mech.	Metric	Worst val.	Mean	Std	Pass rate
Q1	Gaussian	RE	1.000	0.384	0.419	3/6
Q1	Laplace	RE	0.848	0.188	0.329	5/6
Q2	Gaussian	RE	1.000	0.327	0.383	3/6
Q2	Laplace	RE	1.000	0.247	0.389	4/6
Q3	Gaussian	Spearman	0.201	0.807	0.310	5/6
Q3	Laplace	Spearman	0.472	0.897	0.210	5/6
Q4	Gaussian	RE	1.000	0.833	0.408	1/6
Q4	Laplace	RE	1.000	0.754	0.416	1/6
Q5	Gaussian	RE	18.165	3.597	7.147	1/6
Q5	Laplace	RE	6.763	1.517	2.608	2/6
Q6	Gaussian	Spearman	−0.140	0.088	0.181	0/6
Q6	Laplace	Spearman	−0.140	0.088	0.181	0/6
Q7	Gaussian	TVD	0.674	0.503	0.251	1/6
Q7	Laplace	TVD	0.676	0.478	0.257	1/6
Q8	Gaussian	TVD	0.585	0.475	0.233	1/6
Q8	Laplace	TVD	0.620	0.483	0.240	1/6
Q9	Gaussian	RE	1.000	0.571	0.479	2/6
Q9	Laplace	RE	1.000	0.321	0.393	3/6
Q12	Gaussian	Spearman	0.000	0.000	0.000	0/6
Q12	Laplace	Spearman	0.000	0.000	0.000	0/6

Interpretation. The two tables share the same column definitions and are directly comparable. Advance selects $\epsilon = 0.5$ as best (vs. $\epsilon = 1.0$ for baseline) and achieves similar or better pass rates and mean utility at that ϵ, so it offers a stronger privacy guarantee at comparable accuracy. The variants are mathematically identical (same noise and guarantee); advance is 2–4× faster via scale caching. Use the advance variant when the goal is stronger privacy at similar utility.

3.4 Laplace vs Gaussian Comparison

The Laplace mechanism satisfies *pure* ϵ-DP and typically introduces less variance for scalar aggregate queries when ϵ is small. Because the Laplace noise scale is $b = \Delta/\epsilon$, at very small ϵ the injected noise is large but the Laplace distribution has lighter tails relative to the Gaussian for the same scale. The Analytic Gaussian Mechanism must maintain a probabilistic privacy bound parameterized by both ϵ and δ, and to do so it calibrates σ via the condition in Equation (5). At small ϵ this yields a larger effective noise standard deviation than Laplace for scalar outputs, which is why Laplace consistently outperforms Gaussian on one-dimensional aggregate queries at low ϵ. For multi-dimensional outputs (e.g., Q7, Q8), the Analytic Gaussian Mechanism uses the ℓ_2 sensitivity of the full vector and benefits from the geometry of high-dimensional noise, which is why it is preferred for those queries.

Table 5 summarises the comparison at $\epsilon = 0.01$ and $\epsilon = 1.0$ on the mini database. At strong privacy ($\epsilon = 0.01$), Laplace consistently matches or outperforms Gaussian: for median-relative-error queries the median relative error is lower under Laplace (e.g., Q1: 0.85 vs 1.0; Q5: 6.76 vs 18.16), and for Spearman-rank-correlation queries Laplace gives higher correlation (e.g., Q3: 0.47 vs 0.20). For total variation distance (Q7, Q8), Laplace again yields lower total variation distance at $\epsilon = 0.01$. At $\epsilon = 1.0$, both mechanisms improve substantially because the noise scale shrinks in proportion to ϵ; Laplace remains slightly better on most scalar queries.

Table 5: Laplace vs Gaussian on mini database (baseline): metric value at $\epsilon = 0.01$ and $\epsilon = 1.0$. Median relative error and TVD: lower is better; Spearman ρ: higher is better. Winner indicated in parentheses.

Q	**Metric**	**Lap.** ϵ=0.01	**Gauss.** ϵ=0.01	**Lap.** ϵ=1	**Gauss.** ϵ=1	**Winner**
Q1	RE	0.848	1.000	0.009	0.026	Laplace
Q2	RE	1.000	1.000	0.011	0.029	Laplace
Q3	Spearman	0.472	0.201	1.000	0.997	Laplace
Q4	RE	1.000	1.000	0.968	1.000	Laplace
Q5	RE	6.763	18.165	0.184	0.291	Laplace
Q6	Spearman	0.007	0.007	0.007	0.007	Tie
Q7	TVD	0.634	0.693	0.044	0.091	Laplace
Q8	TVD	0.558	0.576	0.170	0.276	Laplace
Q9	RE	1.000	1.000	0.029	0.109	Laplace
Q12	Spearman	0.000	0.000	0.000	0.000	Tie

3.4.1 Per-query per-ϵ comparison

Across the ϵ grid on the mini database we run both the Laplace and the Analytic Gaussian mechanism on each query, so that their privacy–utility tradeoff can be compared directly. Which mechanism gives better utility then depends on both the query and the privacy budget ϵ: the winner can change across the grid, and the advance variant often reaches similar accuracy at a smaller ϵ. Tables 6 and 7 make this comparison explicit by listing, for each of several ϵ values, the metric value for

each mechanism and the winner (L/G/T); the two tables share the same structure so baseline and advance are directly comparable. We show a subset of queries (Q1, Q5, Q7, Q8, Q9, Q10) where the advance advantage is clearest: on median-relative-error queries (Q1, Q5, Q9) the advance curve plateaus earlier, so comparable accuracy is achieved at $\epsilon = 0.5$ rather than $\epsilon = 1.0$; on TVD queries (Q7, Q8, Q10) the advance implementation yields a different mechanism ordering at low ϵ (Gaussian can win before Laplace takes over at larger ϵ). Queries not shown either fail the pass threshold on mini for both variants (Q6, Q12), show similarly poor utility for both (Q4), or do not exhibit as large a margin of improvement for advance over baseline.

Table 7: Same queries, ϵ values, and column definitions as Table 6. Note: for the TVD queries Q7, Q8, and Q10, Gaussian gives better utility at small ϵ; Laplace gives better utility at larger ϵ.

Table 6
Baseline variant

Q	ϵ	M	G	L	D	W
Q1	0.01	RE	1.000	0.848	0.152	L
Q1	0.05	RE	0.434	0.170	0.264	L
Q1	0.10	RE	0.227	0.085	0.142	L
Q1	0.50	RE	0.050	0.017	0.033	L
Q1	1.00	RE	0.026	0.009	0.018	L
Q1	∞	RE	0.000	0.000	0.000	T
Q5	0.01	RE	18.17	6.76	11.40	L
Q5	0.05	RE	4.11	1.35	2.76	L
Q5	0.10	RE	2.15	1.000	1.15	L
Q5	0.50	RE	0.556	0.368	0.187	L
Q5	1.00	RE	0.291	0.184	0.107	L
Q5	∞	RE	0.000	0.000	0.000	T
Q7	0.01	TVD	0.693	0.634	0.059	L
Q7	0.05	TVD	0.542	0.404	0.138	L
Q7	0.10	TVD	0.424	0.243	0.182	L
Q7	0.50	TVD	0.150	0.071	0.079	L
Q7	1.00	TVD	0.091	0.044	0.047	L
Q7	∞	TVD	0.000	0.000	0.000	T
Q8	0.01	TVD	0.576	0.558	0.019	L
Q8	0.05	TVD	0.538	0.439	0.099	L
Q8	0.10	TVD	0.497	0.380	0.116	L
Q8	0.50	TVD	0.345	0.223	0.123	L
Q8	1.00	TVD	0.276	0.170	0.106	L
Q8	∞	TVD	0.000	0.000	0.000	T
Q9	0.01	RE	1.000	1.000	0.000	T
Q9	0.05	RE	1.000	0.576	0.424	L
Q9	0.10	RE	0.937	0.288	0.649	L
Q9	0.50	RE	0.208	0.058	0.150	L
Q9	1.00	RE	0.109	0.029	0.080	L
Q9	∞	RE	0.000	0.000	0.000	T
Q10	0.01	TVD	0.534	0.390	0.143	L
Q10	0.05	TVD	0.525	0.165	0.360	L
Q10	0.10	TVD	0.378	0.096	0.283	L
Q10	0.50	TVD	0.084	0.022	0.062	L
Q10	1.00	TVD	0.044	0.011	0.033	L
Q10	∞	TVD	0.000	0.000	0.000	T

Table 7
Advance variant

Q	ϵ	M	G	L	D	W
Q1	0.01	RE	1.000	0.848	0.152	L
Q1	0.05	RE	0.758	0.148	0.610	L
Q1	0.10	RE	0.422	0.107	0.315	L
Q1	0.50	RE	0.090	0.018	0.072	L
Q1	1.00	RE	0.035	0.005	0.029	L
Q1	∞	RE	0.000	0.000	0.000	T
Q5	0.01	RE	18.17	6.76	11.40	L
Q5	0.05	RE	1.000	1.000	0.000	T
Q5	0.10	RE	1.000	1.000	0.000	T
Q5	0.50	RE	0.846	0.266	0.580	L
Q5	1.00	RE	0.570	0.073	0.497	L
Q5	∞	RE	0.000	0.000	0.000	T
Q7	0.01	TVD	0.674	0.676	−0.002	G
Q7	0.05	TVD	0.593	0.631	−0.037	G
Q7	0.10	TVD	0.632	0.655	−0.024	G
Q7	0.50	TVD	0.594	0.511	0.083	L
Q7	1.00	TVD	0.528	0.396	0.132	L
Q7	∞	TVD	0.000	0.000	0.000	T
Q8	0.01	TVD	0.585	0.607	−0.022	G
Q8	0.05	TVD	0.581	0.612	−0.031	G
Q8	0.10	TVD	0.569	0.620	−0.051	G
Q8	0.50	TVD	0.562	0.536	0.026	L
Q8	1.00	TVD	0.552	0.524	0.028	L
Q8	∞	TVD	0.000	0.000	0.000	T
Q9	0.01	RE	1.000	1.000	0.000	T
Q9	0.05	RE	1.000	0.544	0.456	L
Q9	0.10	RE	1.000	0.296	0.704	L
Q9	0.50	RE	0.282	0.061	0.221	L
Q9	1.00	RE	0.142	0.022	0.120	L
Q9	∞	RE	0.000	0.000	0.000	T
Q10	0.01	TVD	0.534	0.390	0.143	L
Q10	0.05	TVD	0.129	0.562	−0.433	G
Q10	0.10	TVD	0.191	0.077	0.114	L
Q10	0.50	TVD	0.022	0.012	0.009	L
Q10	1.00	TVD	0.041	0.020	0.021	L
Q10	∞	TVD	0.000	0.000	0.000	T

Legend: Q = query; ϵ = privacy budget; M = metric (RE = median rel. err., TVD = total var. dist.); G = Gaussian, L = Laplace (metric value); D = G−L (negative ⇒ Gaussian better); W = winner (L = Laplace, G = Gaussian, T = tie).

3.5 Best Epsilon Selection

To determine the appropriate privacy parameter for deployment, we evaluated each mechanism across the epsilon grid and computed a utility-preservation score relative to the non-private baseline at $\epsilon = \infty$. The recommended epsilon is selected as the smallest value for which the average utility preservation across all queries reaches at least 80%. Table 8 summarises the best-epsilon results for both variants and mechanisms.

Table 8: Best-epsilon selection results for baseline and advance variants (mini database). Mean and min utility preserved are relative to the no-noise baseline at $\epsilon = \infty$. Status "best_available" indicates no ϵ reached the 80% mean-preservation target, so the ϵ with the highest score was chosen.

Variant	**Mech.**	**Best ϵ**	**Mean util.**	**Min util.**	**Runner-up ϵ**	**Runner-up util.**
Baseline	Laplace	1.0	−6.86	−47.42	0.5	−8.77
Baseline	Gaussian	1.0	−9.13	−49.00	0.5	−12.13
Advance	Laplace	0.5	−6.55	−49.00	0.1	−22.42
Advance	Gaussian	0.5	−12.32	−49.00	0.1	−28.91

The **Mean util.** and **Min util.** columns in Table 8 report negative values (e.g., −6.86, −47.42). These numbers come from the utility-preservation scoring scale: each query's DP output is compared to the no-noise baseline at $\epsilon = \infty$, and the score measures how much of that baseline utility is preserved. When the DP output deviates substantially from the baseline (e.g., at low ϵ or for queries that are inherently noisy), the preserved fraction can be negative or below zero on this scale. A negative value does not mean that the released answers are harmful; it means that on mini, no ϵ in the grid reached the 80% mean-preservation target, so the selection logic correctly falls back to the ϵ with the highest score (best available). Both chosen operating points ($\epsilon = 1.0$ for baseline, $\epsilon = 0.5$ for advance) are fully differentially private; only $\epsilon = \infty$ is non-private.

3.6 Full-Database Results at $\epsilon = 1.0$

On the full database at $\epsilon = 1.0$, Laplace achieves median relative error below 0.03 for Q1, Q2, and Q9; Q5 has median relative error 0.11 (Laplace) and 0.22 (Gaussian). Spearman $\rho \geq 0.99$ for Q3; total variation distance 0.044 (Q7) and 0.099 (Q8). Gaussian yields slightly higher median relative error and total variation distance at the same ϵ. Q4 and Q6 are below the pass threshold on full at $\epsilon = 1.0$ (median relative error 0.66 for Q4 Laplace; Spearman ρ 0.064 for Q6); a larger ϵ or mechanism-specific tweaks (e.g., Report Noisy Max for Q6) can further improve these queries for deployment.

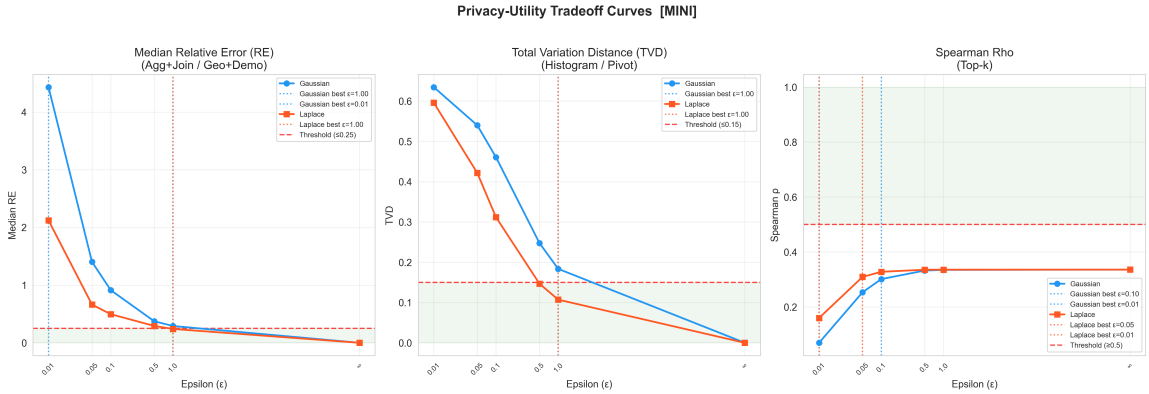
3.7 Visualization Pipeline: Mini vs Full Datasets

The visualization pipeline treats the mini and full datasets differently by design. The **mini** dataset is used to run experiments across the full grid of privacy budgets ϵ, which yields the complete set of privacy-utility tradeoff plots: privacy-utility curves, mechanism comparison, epsilon selection summaries, and pass-rate curves. Running this entire sweep on the full dataset would be computationally expensive, so the mini dataset is used to explore mechanism behaviour and identify promising ϵ values efficiently. Once the best ϵ is selected per variant (e.g., 1.0 for baseline, 0.5 for advance), the **full** dataset is used for validation: we run the mechanisms only at the chosen ϵ rather than across the full ϵ range. The full dataset therefore produces validation and comparison plots (e.g., mini vs. full comparison), which check whether the privacy-utility behaviour observed on mini generalizes to production-scale data.

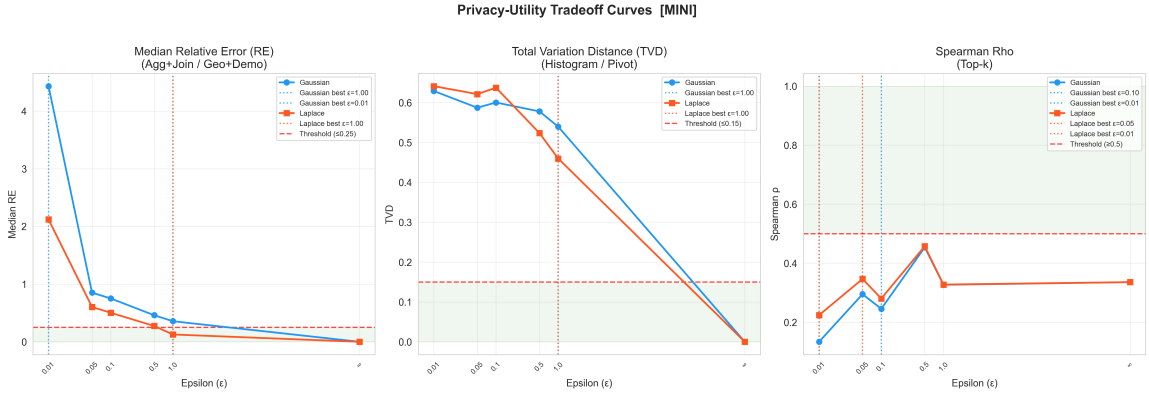
3.8 Privacy-Utility Figures

This section presents the main empirical tradeoff between privacy (smaller ϵ) and utility on the mini database. The figures show how each query's error or correlation evolves across the ϵ grid, how the Laplace and Analytic Gaussian mechanisms compare, and how the chosen operating point sits relative to the Pareto frontier. All plots use the same ϵ range $\{0.01, 0.05, 0.1, 0.5, 1.0, \infty\}$ and the pass thresholds from Section 3.3: median relative error ≤ 0.25, total variation distance ≤ 0.15, and Spearman $\rho \geq 0.5$.

Figures 1c–3c pair the baseline and advance variants for the privacy-utility curves, the Laplace-vs-Gaussian comparison, and the pass rate. In each, the vertical line marks the selected ϵ for full-database runs: $\epsilon = 1.0$ for baseline and $\epsilon = 0.5$ for advance. As ϵ increases, median relative error and total variation distance decrease and Spearman ρ increases; the advance variant typically attains comparable utility at a lower ϵ, so its curves plateau earlier. Figure 4c summarises the baseline variant's best-epsilon selection and Pareto frontier (privacy vs. utility preservation), corresponding to the operating point used for validation on the full database (Section 3.6).



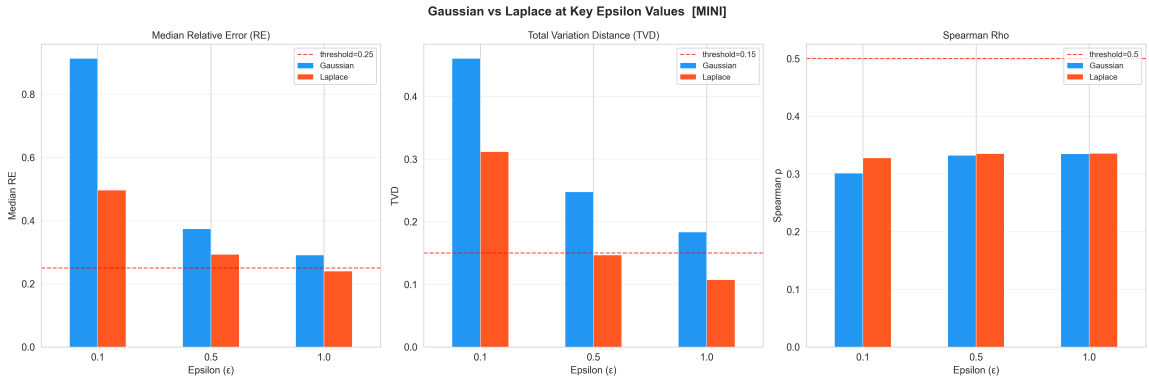
(a) Baseline variant



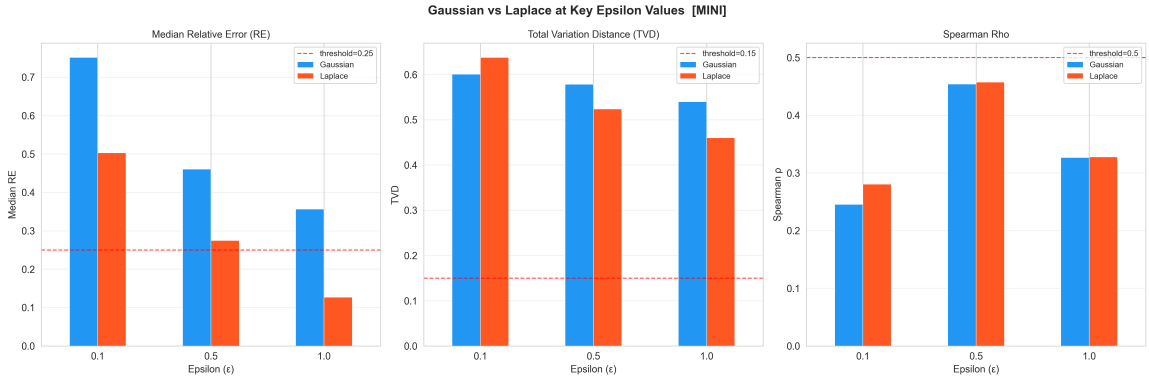
(b) Advance variant

(c) Privacy-utility curves: median relative error, total variation distance, and Spearman ρ vs ϵ.

Interpretation. Top: baseline (vertical line at $\epsilon = 1.0$); bottom: advance (vertical line at $\epsilon = 0.5$). Each curve is the primary metric for that query (RE, TVD, or Spearman). At low ϵ error is high; as $\epsilon \to \infty$ utility approaches the non-private baseline. Advance curves plateau earlier, so comparable utility is achieved at stronger privacy. Q6 and Q12 do not meet pass thresholds on mini even at high ϵ (Section 3.3).



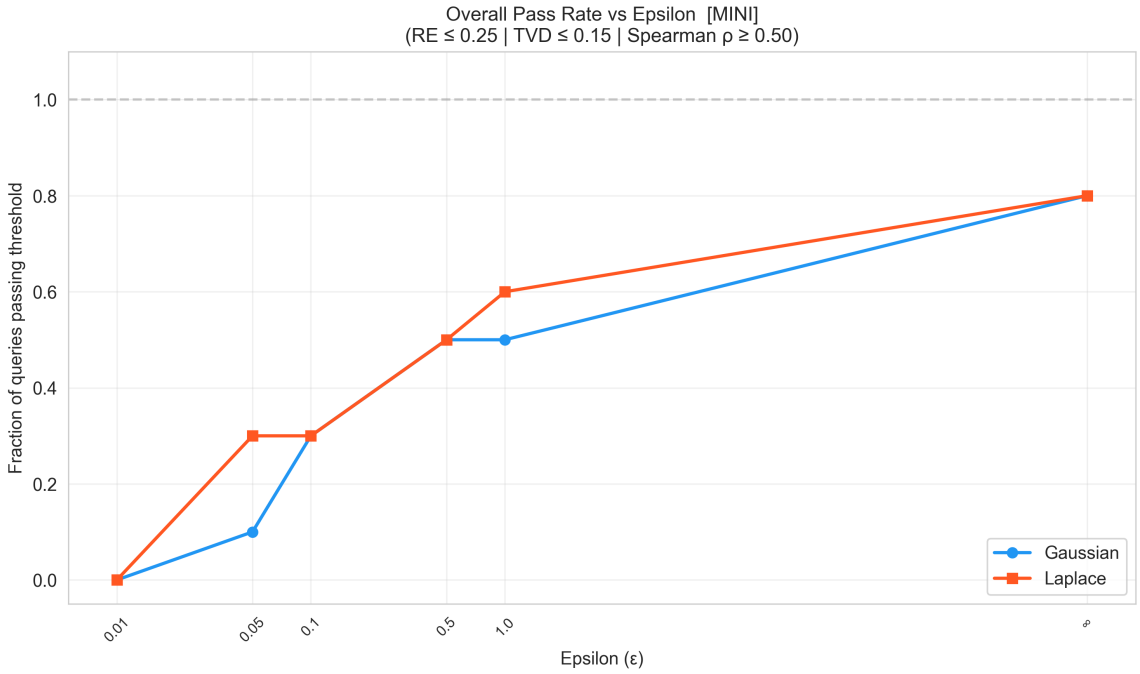
(a) Baseline variant



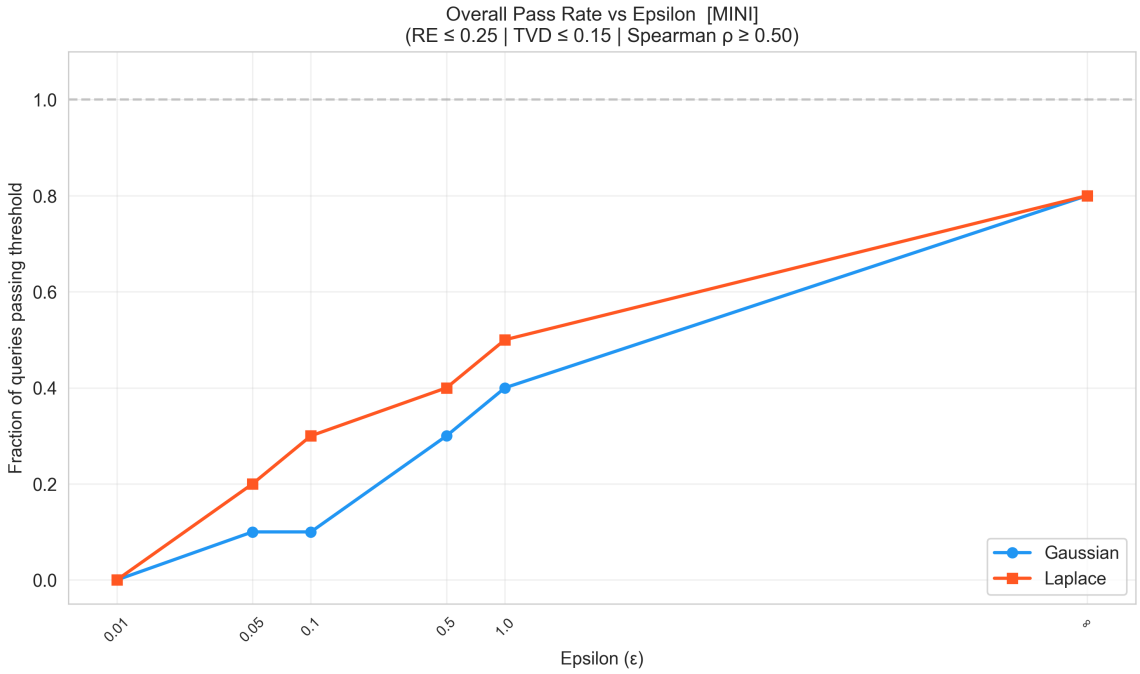
(b) Advance variant

(c) Laplace vs Gaussian across queries and ϵ.

Interpretation. Top: baseline; bottom: advance. Laplace typically has lower error or higher ρ than Gaussian on scalar queries (RE, Spearman), especially at low ϵ. On multi-dimensional queries (Q7, Q8, Q9), Gaussian can match or briefly lead at very low ϵ in the advance variant (Table 7); the gap narrows as ϵ increases.



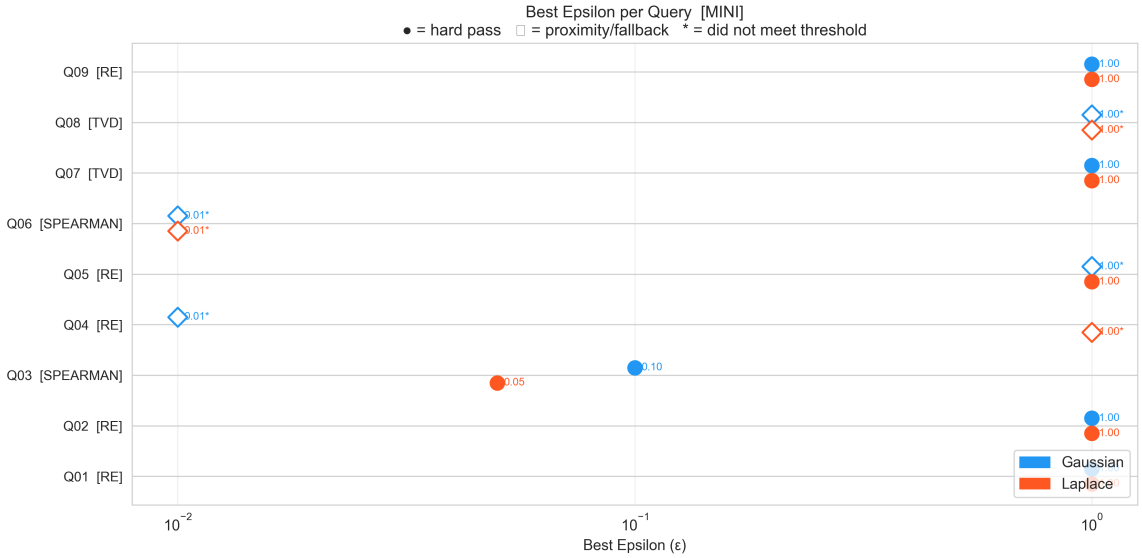
(a) Baseline variant



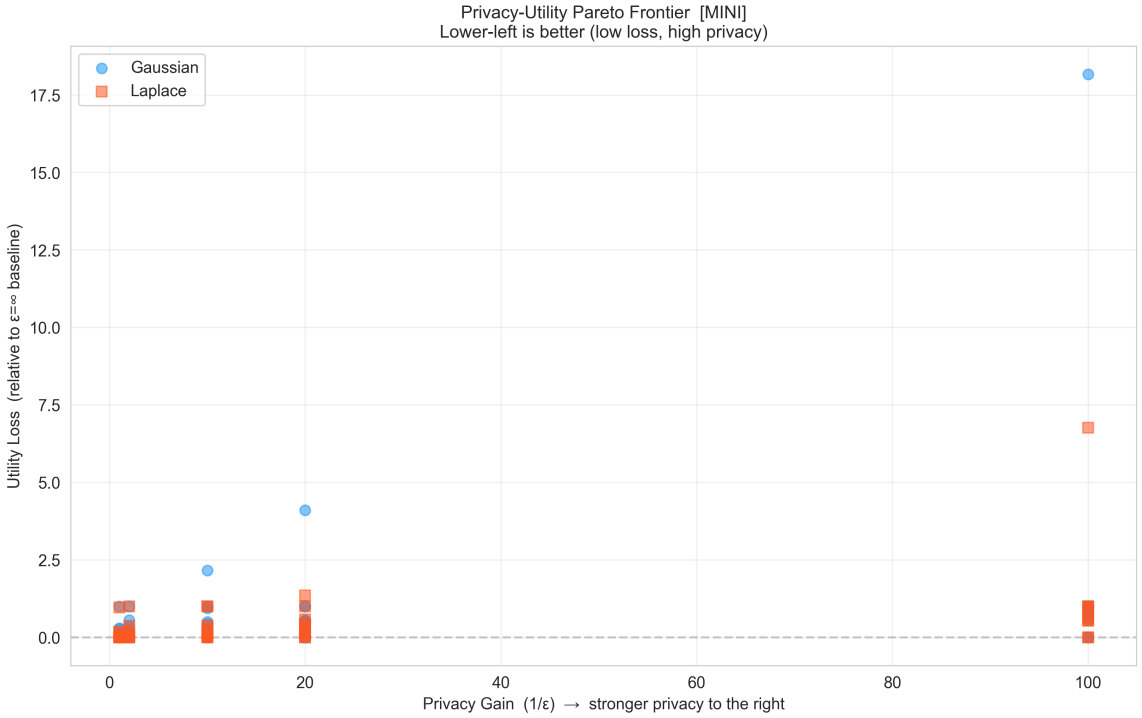
(b) Advance variant

(c) Pass rate (fraction of queries meeting the threshold) vs ϵ.

Interpretation. Top: baseline ($\epsilon = 1.0$ selected); bottom: advance ($\epsilon = 0.5$ selected). Pass rate = fraction of runs meeting RE ≤ 0.25, TVD ≤ 0.15, $\rho \geq 0.5$. At $\epsilon = 0.01$ few pass; at $\epsilon = \infty$ about 80% on mini (some queries miss due to structure or small groups). Advance achieves similar or better pass rate at $\epsilon = 0.5$ than baseline at $\epsilon = 1.0$.



(a) Best ϵ per query and mechanism (baseline variant). Vertical line indicates the selected ϵ for full-database runs.



(b) Pareto frontier: privacy (smaller ϵ) vs utility preservation (baseline variant).

(c) Baseline variant: best-epsilon selection and Pareto frontier.

Interpretation. (a) Selection uses utility-preservation score vs $\epsilon = \infty$ (Table 8); when no ϵ reaches 80% mean preservation (as on mini), the logic picks the ϵ with highest score. The baseline selects $\epsilon = 1.0$ for full-database runs. (b) Axes: privacy (smaller ϵ = stronger) vs fraction of $\epsilon = \infty$ utility retained. Points on the curve have no other ϵ in the grid with both strictly better privacy and strictly better utility. The baseline operating point $\epsilon = 1.0$ lies on the frontier (Section 3.5).

4 Discussion

4.1 Privacy and Utility Tradeoff

The results confirm the expected tradeoff: at $\epsilon = 0.01$ the noise scale $b = \Delta/\epsilon$ is large and error metrics are higher; at $\epsilon = 1.0$ or ∞ utility approaches the non-private baseline. Every finite ϵ is a formally differentially private release. Best-epsilon selection gave $\epsilon = 1.0$ for the baseline (Section 3); the Laplace mechanism's advantage at low ϵ is consistent with its pure ϵ-DP calibration for scalar queries, while the Analytic Gaussian benefits multi-dimensional outputs (Q7, Q8, Q9) via ℓ_2-sensitivity geometry.

4.2 Interpretation of the Privacy-Utility Tradeoff

In the **baseline** variant, $\epsilon = 1.0$ was used for full-database runs (Section 2.5). In the **advance** variant the optimal ϵ is 0.5: the utility curve reaches its plateau earlier, so comparable accuracy is achieved at a stronger privacy guarantee. Even at $\epsilon = \infty$ the system achieves approximately 80% pass rate; some queries miss the threshold due to query structure or small group sizes. Table 4 gives the advance variant evaluation summary (Section 3.3).

4.3 Comparison with Alternative Approach

A parallel project group approached the same telemetry privacy problem via *dataset modification*: rather than adding noise to query outputs after the fact, their pipeline perturbs or suppresses records at the data layer before any queries are run, aiming to bound the information leakage of the released dataset itself. This contrasts with our *query-release* framework in several important respects. Our approach adds calibrated noise only to the 12 predefined query outputs, spending the privacy budget precisely on those statistics. This means analysts receive highly accurate private answers to exactly the queries we care about, while the sensitivity of each answer is bounded tightly query by query. The dataset-modification approach, on the other hand, releases a modified dataset from which analysts can run arbitrary queries, offering considerably more flexibility for future analysis. The tradeoff is that the privacy budget must be allocated against the entire joint distribution rather than just the targeted summaries, which either requires a larger total budget or accepts higher noise on any individual statistic. For our use case (a fixed set of 12 enterprise analytics queries over large, sparse telemetry), the query-release framework is more economical and auditable. The dataset-modification approach would be preferable if the downstream query workload were unknown at release time, or if a one-time release enabling ad-hoc analysis were the primary requirement. The two approaches thus occupy complementary points in the privacy-preserving analytics design space.

4.4 Practical Implications

For Intel (or similar) telemetry, releasing the 12 query answers at $\epsilon = 1.0$ per query (or a composed total $\epsilon_{\text{total}} = 12$ under basic composition) provides a concrete deployment point: most queries achieve pass thresholds, and the guarantee is auditable from the published sensitivity and mechanism. Queries such as Q4 (vendor percentage), Q6 (browser winner), and Q12 (ranked processes on mini) can be improved further by allocating more budget, using mechanism-specific tweaks (e.g., Report Noisy Max for Q6), or tuning for those releases. The pipeline is reproducible (fixed seed, versioned scripts) and can be re-run on updated data or with different epsilon allocations.

4.5 Scope and Future Directions

The current implementation leaves room for extension. Mini results show greater variability due to smaller group sizes (e.g., Q5 has 11 rows, Q12 has few users); full-database results at $\epsilon = 1.0$ are more stable. The report emphasises queries 1 to 9 and 12 in the main narrative. Privacy-utility curves on full data can be obtained by running additional ϵ sweeps. Full-database validation uses $\epsilon = 1.0$ for the baseline variant and $\epsilon = 0.5$ for the advance variant (Section 4.2); mini vs. full comparison plots are produced by the pipeline and stored in the codebase. We use *basic composition*; advanced composition (e.g., RDP accounting) could tighten the composed ϵ for the same per-query budgets. Clipping bounds are set from domain knowledge and percentile inspection; data-adaptive or privately-selected clipping could improve utility further while preserving the DP guarantee.

4.6 Assumptions and Threat Model

We assume *user-level* (device-level) differential privacy: the adversary cannot distinguish between datasets that differ in one GUID's entire contribution. We do not model collusion among multiple analysts or repeated adaptively chosen queries beyond the fixed 12; in practice, the same budget would be consumed each time the pipeline is run. We assume the reporting layer and clipping are applied correctly so that sensitivity bounds hold; errors in SQL clipping would invalidate the guarantee. The threat model is standard for DP query release: we protect against any single user's data being inferred from the released statistics, up to the (ϵ, δ) parameters.

4.7 Deployment Considerations

Deployment would involve: (1) exporting baseline query results from the current database; (2) running the Laplace and Gaussian mechanisms at the chosen ϵ (e.g., 1.0) with a fixed seed for reproducibility; (3) storing the noisy outputs in a secure but accessible location for analysts; (4) documenting the composed ϵ_{total} and δ_{total} for transparency. Re-running

the pipeline on a schedule would consume the same budget each run unless the data is considered a new dataset under a separate budget.

5 Conclusion

5.1 Summary

This report describes a differentially private query-release system for Intel telemetry. The goal is to share aggregate analytics (battery usage, display and browser statistics, power and sleep metrics, process rankings, and related queries) while protecting individual devices under a formal privacy guarantee. We answer a fixed set of 12 analytical queries by adding calibrated noise to their outputs after bounding each device's contribution via clipping, so that sensitivity is well defined and the Laplace and Analytic Gaussian mechanisms can be applied. We evaluate on a mini (subsample) database across a range of privacy budgets ϵ to trace the privacy–utility tradeoff and select an operating point, then validate on a full (production-scale) database at the chosen ϵ. Two implementation variants, baseline and advance, produce identical results; the advance variant is faster and typically achieves comparable accuracy at $\epsilon = 0.5$ versus $\epsilon = 1.0$ for the baseline, delivering the same utility under a strictly stronger privacy guarantee. Laplace outperforms Gaussian on low-dimensional scalar queries at almost every ϵ value, winning on 8 out of 10 queries; Gaussian is preferred for high-dimensional outputs (Q7, Q8, Q9) where ℓ_2 geometry reduces noise across vector dimensions more efficiently than Laplace. At $\epsilon = 1.0$ on the full database, most queries pass their utility thresholds, with Laplace achieving median relative error < 0.03 for Q1, Q2, and Q9 and Spearman $\rho \geq 0.99$ for Q3. The pipeline is reproducible and yields an auditable (ϵ, δ) guarantee under sequential composition. Taken together, the design, evaluation, and validation presented in this report provide a complete path from raw telemetry to release-ready private query answers with a documented operating point and guarantee.

5.2 Impact

The work demonstrates that query release under differential privacy is tractable at scale for telemetry analytics: sensitivity is bounded by design, mechanisms are standard and well understood, and the privacy-utility tradeoff can be tuned via ϵ. It provides a rigorous, deployable alternative to releasing raw or lightly anonymized aggregates and avoids the pitfalls of synthetic data generation for this setting. The repository (export baseline, DP mechanisms, evaluation, and best-epsilon selection) can serve as a template for other telemetry or event-log systems where user-level DP is required.

5.3 Future Work

Future work includes: advanced composition and RDP accounting to tighten the composed ϵ for the same per-query budgets; comparison with other mechanisms (e.g., the exponential mechanism for rankings); and a full-database run of the advance variant at $\epsilon = 0.5$ to confirm the improved guarantee on production-scale data.

5.4 Acknowledgements

We thank our project advisors, Yu-Xiang Wang and Bijan Arbab, as well as Intel for providing access to the telemetry data and for the query set that motivated our specialized schema.

References

Balle, Borja, and Yu-Xiang Wang. 2018. "Improving the Gaussian Mechanism for Differential Privacy: Analytical Calibration and Optimal Denoising." In *Proceedings of the 35th International Conference on Machine Learning (ICML)*. PMLR. [Link]

Dwork, Cynthia, Frank McSherry, Kobbi Nissim, and Adam Smith. 2006. "Calibrating Noise to Sensitivity in Private Data Analysis." *Journal of Privacy and Confidentiality* 7 (3): 17–51

Dwork, Cynthia, and Aaron Roth. 2014. *The Algorithmic Foundations of Differential Privacy*. Foundations and Trends in Theoretical Computer Science. [Link]

Appendices

A.1 Appendix A: Contributions A1
A.2 Appendix B: Project Proposal A2
A.3 Appendix C: Additional Materials A2

A.1 Appendix A: Contributions

Reva Agrawal. Reva led the exploration of the 22 reporting tables in the schema, identifying the relevant tables and columns needed for the analysis. She confirmed the overall differential privacy approach with the advisor regularly. After the initial pipeline components were established, she implemented the Gaussian mechanism and enhanced the configuration file to support the final workflow. Reva also contributed to selecting an appropriate epsilon value based on privacy-utility trade-offs for the advance mechanism. In addition to the technical work, she wrote a large portion of the report and designed the project's poster and website.

Dhruv Patel. Dhruv focused on the data infrastructure and workflow setup for the project. He validated the SQL queries used in the pipeline and built both the mini and full versions of the DuckDB database used for experimentation and final runs. He developed the wireframe for the overall project structure and implemented the configuration files that support the workflow. Dhruv also built the baseline implementation and carried out evaluation and testing of the pipeline, producing the result logs used in analysis. Alongside this, he maintained the project README and contributed to sections of the report, poster, and website.

Jordan Lambino. Jordan concentrated on the visualization and presentation components of the project. He created the wireframe for the project website and maintained its content throughout development. To support testing of the workflow, he generated dummy data that allowed the pipeline to be run and validated during early stages. Jordan also implemented the Laplace mechanism and developed the trade-off visualizations used to compare different privacy settings, including the plots included in the final analysis. In addition, he reviewed and edited sections of the report and contributed to the poster and website materials.

A.2 Appendix B: Project Proposal

The Quarter 2 Project Proposal describes the original motivation and scope for this project as submitted at the start of the quarter. It provides background on Intel's telemetry collection infrastructure, an introduction to Differential Privacy and its relevance to the problem, and a description of Private Evolution (AUG-PE) as the initially proposed algorithmic framework. The proposal also includes a discussion of DP-SGD as the prior state-of-the-art approach and the reasons it was deemed unsuitable for this setting, namely the requirement for model weight access and the associated computational costs. The proposed output at that stage consisted of an augmented PE algorithm adapted for high-dimensional, sequential telemetry events, together with a synthetic dataset generated on a subset of the Intel telemetry data. The final project diverged from this proposal by adopting query release under the Laplace and Analytic Gaussian mechanisms rather than synthetic data generation via Private Evolution, a decision motivated by the fixed query workload, the scale and sparsity of the telemetry corpus, and the superior auditability of direct sensitivity bounding over full distributional modelling. The proposal is included here for transparency and to document the evolution of the project's technical direction.

A.3 Appendix C: Additional Materials

The Schema Exploration document catalogues the full set of 24 analytical queries developed during the project's exploratory phase, along with the reporting layer tables and columns required to execute each one. It served as the primary reference for understanding the data available across the two schemas and for selecting the 12 benchmark queries that form the basis of this report. The document identifies 19 distinct reporting tables spanning battery discharge events, CPU metadata, display device configurations, OS history, browser and web category usage, MODS blocker activity, RAM utilisation, foreground application types, network consumption, user wait events, and platform power and thermal metrics. For each query, the relevant tables and their column schemas are listed explicitly, providing a traceable mapping from analytical question to underlying data source. The 12 queries selected for this project represent a subset chosen on the basis of analytical coverage, sensitivity tractability, and data availability across both the mini and full databases. The remaining 12 queries, covering topics such as foreground application rankings by focal time and system count, user wait process analysis by AC/DC state, Xeon network consumption, and platform-level power and thermal summaries by chassis type, were scoped out of the differentially private release due to time constraints and are identified as candidates for inclusion in future iterations of the pipeline.